

Reviewer Pack — Minimal Quadratic Surface Area and Frege Proof Width Inequal...

Entry 57741a1bc97e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 11:51:41 UTC

Minimal Quadratic Surface Area and Frege Proof Width Inequality

Entry ID: 57741a1bc97e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 11:51:41 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Surfaces) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For every Boolean satisfiability instance φ , the minimal quadratic surface area of its associated quadratic form is linearly correlated with its Frege proof width $w(\varphi)$, such that the surface area $A(\varphi) = \Theta(w(\varphi))$ and the volume $V(\varphi) \leq 4 * A(\varphi)$.

Rationale (proposer's reasoning):

Quadratic surfaces have been studied in algebraic geometry, but their application to complexity theory is rare. The quadratic surface area of a form can be computed algorithmically, providing a novel geometric invariant for proving lower bounds on proof complexities. If this invariant correlates with Frege proof width, it could reveal unexpected connections between geometry and computational complexity.

Taxonomy category: AlgebraicGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 63ebe7b883980249

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal quadratic surface area $A(\varphi)$ and Frege proof width $w(\varphi)$ is ≥ 0.8 for all seeds, and the ratio of volume $V(\varphi)$ to $A(\varphi)$ is ≤ 1.25 for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - surface area AND quadratic surfaces AND Boolean satisfiability AND Frege proof width - algebraic geometry AND quadratic forms AND related to Boolean satisfiability problems - Frege proof complexity AND linear relationship with quadratic surface volume and area

Top relevant hits considered: - [http://arxiv.org/abs/2604.08095v2] The Boolean surface area of polynomial threshold functions - [http://arxiv.org/abs/cond-mat/9604012v1] Incomplete melting of the Si(100) surface from molecular-dynamics simulations using the Effective-Medium Tight-Binding m - [http://arxiv.org/abs/cond-mat/0110236v1] Applicability of the Broken-Bond Rule to the Surface Energy of the fcc Metals - [http://arxiv.org/abs/2605.24303v1] Geometry of Neighborhoods of Minimal Rational Curves - [http://arxiv.org/abs/1907.00024v2] Curve counting in genus one: elliptic singularities & relative geometry - [http://arxiv.org/abs/math/0205005v2] The rings of noncommutative projective geometry - [http://arxiv.org/abs/2402.10351v3] Quantum Automating $\mathbf{TC}^0$ -Frege Is LWE-Hard - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.9s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2**n - 1):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            cnf.append(clause)
        return cnf

    def compute_quadratic_form(cnf):
        n = len(cnf[0])
        Q = [[0] * n for _ in range(n)]
        for clause in cnf:
            for literal in clause:
                var = abs(literal) - 1
                Q[var][var] += 1
        return Q

    def min_quadratic_surface_area(Q):
```

```

    n = len(Q)
    area = 0
    for i in range(n):
        for j in range(i + 1, n):
            area += abs(Q[i][j])
    return area

def volume(A):
    return A ** (3/2) / math.gamma(5/2)

def frege_proof_width(cnf):
    # Placeholder function; actual implementation needed
    return len(cnf)

instances_tested = 0
n_max = 0
total_area = 0
total_volume = 0
total_width = 0

for n in [5, 10, 15, 20, 30, 40]:
    cnf = generate_cnf(n)
    Q = compute_quadratic_form(cnf)
    area = min_quadratic_surface_area(Q)
    volume_val = volume(area)
    width = frege_proof_width(cnf)

    total_area += area
    total_volume += volume_val
    total_width += width

    instances_tested += 1
    n_max = max(n_max, n)

mean_area = total_area / instances_tested
mean_volume = total_volume / instances_tested
mean_width = total_width / instances_tested

correlation_coefficient = (instances_tested * mean_area * mean_width -
                           sum(area * width for area, width in zip([mean_area] * instances_tested,
                                                                     [mean_width] * instances_tested)) /
                           math.sqrt((instances_tested * mean_area**2 - sum(area**2 for area in [mean_area] * instances_tested)) *
                                       (instances_tested * mean_width**2 - sum(width**2 for width in [mean_width] * instances_tested))))

conjecture_holds = correlation_coefficient >= 0.8 and mean_volume <= 4 * mean_area

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Correlation coefficient: {correlation_coefficient}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

```

```

results = []

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and max(r["n_max"] for r in results) >= 1:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient or Volume/Areas ratio of {first_failing_seed}\"")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_data_or_budget_exceeded n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support conditions could not be evaluated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14091
2	preregistration	ollama_remote	glm4:latest	0	0	9414
3	novelty	ollama_remote	glm4:latest	0	0	8959
4	novelty	ollama_remote	glm4:latest	0	0	10534
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	91053
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9063
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11537
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51150

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	75318

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 281118 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/57741a1bc97e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/57741a1bc97e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/57741a1bc97e.tar.gz (if generated)